Spring Framework Annotations with Examples

1. @Component

```
@Component
Marks a class as a Spring-managed bean.
Example:
@Component
public class NotificationService {
    public void send(String message) {
        System.out.println("Sending: " + message);
    }
}
```

2. @Autowired and @Qualifier

```
@Autowired injects dependencies; @Qualifier specifies the bean name.
Example:
@Component
public class UserService {
    @Autowired
    @Qualifier("notificationService")
    private NotificationService notificationService;
}
```

3. @Configuration and @Bean

```
@Configuration defines a config class; @Bean registers a bean.
Example:
@Configuration
public class AppConfig {
    @Bean
    public NotificationService notificationService() {
        return new NotificationService();
    }
}
```

4. @Value

```
@Value injects values from application.properties.
Example:
```

```
@Value("${app.name}")
private String appName;
```

5. @RestController and @GetMapping

```
@RestController marks REST controllers; @GetMapping maps HTTP GET.
Example:
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, World!";
    }
}
```

6. @RequestParam and @PathVariable

```
Binds query parameters and URI variables.
Example:
@GetMapping("/greet")
public String greet(@RequestParam String name) {
    return "Hello, " + name;
}

@GetMapping("/user/{id}")
public String getUser(@PathVariable int id) {
    return "User ID: " + id;
}
```

7. @SpringBootApplication

```
Main entry point for Spring Boot apps.
Example:
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Spring Framework Annotations with Examples

8. @EnableWebSecurity and @PreAuthorize

```
Enables web security and method-level security.
Example:
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests().anyRequest().authenticated();
    }
}

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
```

9. @Entity and @Repository

```
Defines JPA entity and repository.
Example:
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
}

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    User findByName(String name);
}
```

10. @Aspect and @Before

```
Defines aspects for cross-cutting concerns.
Example:
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.example.service.*.*(..))")
```

```
    public void logMethodCall(JoinPoint joinPoint) {
        System.out.println("Calling: " + joinPoint.getSignature());
    }
}
```

11. @SpringBootTest and @MockBean

Used for Spring Boot tests with mocks.

Example:

```
@SpringBootTest
public class UserServiceTest {
    @MockBean
    private UserRepository userRepository;

    @Autowired
    private UserService userService;

    @Test
    public void testFindUser() {
        when(userRepository.findByName("John")).thenReturn(new User("John"));
        assertEquals("John", userService.findUser("John").getName());
    }
}
```